

CRUDIER CRM

Engineering Task Distribution — Sprint 1

Issued: May 27, 2026 • Team: 10 Engineers • Confidential

#	Task ID	Engineer	Module	Priority	Deadline
1	CRUD-001	Arjun Sharma	Project Initialization & Backe	Critical	Day 3
2	CRUD-002	Priya Nair	Authentication & Role-Based Ac	Critical	Day 5
3	CRUD-003	Rohan Mehta	Workspace / Startup Management	High	Day 7
4	CRUD-004	Sneha Kulkarni	Team Member Management	High	Day 8
5	CRUD-005	Vikram Reddy	Task Management System	Critical	Day 10
6	CRUD-006	Anjali Desai	Attendance Management System	High	Day 11
7	CRUD-007	Karan Joshi	Notes & Documentation Module	Medium	Day 12
8	CRUD-008	Divya Iyer	Meeting & Scheduling Module	Medium	Day 13
9	CRUD-009	Nikhil Bose	Notification System	High	Day 14
10	CRUD-010	Meera Pillai	Sales CRM Module	High	Day 16

Prompt 1 — Project Initialization & Backend Setup

Initialize Node.js / Express Backend Foundation

Set up the complete backend skeleton for Crudier CRM using Node.js and Express. Scaffold a production-ready folder structure separating routes, controllers, services, models, middlewares, utils, and config. Install only the approved packages: express, mongodb, dotenv, cors, bcryptjs, jsonwebtoken, cookie-parser, multer, morgan, helmet, and express-rate-limit. No additional dependencies without team-lead approval.

Configure MongoDB Atlas using the provided connection string. The connection utility must handle reconnection, log status, and export a reusable db client. Store all sensitive values — URI, JWT secrets, port — in .env and provide a fully documented .env.example.

Wire app.js and server.js. Apply global middleware in order: morgan, helmet, cors, body parsers, cookie-parser, and a global rate limiter (100 req/15 min). Add a centralized error handler that returns structured JSON. Implement an asyncWrapper utility to eliminate repetitive try/catch blocks. Version all routes under /api/v1. Expose a GET /api/v1/health endpoint returning server status, uptime, and DB connectivity. Write dev, start, and lint npm scripts, and document setup in README.md.

Key Deliverables

✓ Scaffolded folder structure committed to repo
✓ .env.example with all variables documented
✓ MongoDB Atlas connection utility with error handling
✓ Global middleware stack wired in correct order
✓ asyncWrapper + centralized error handler
✓ GET /api/v1/health endpoint
✓ README with setup instructions

Build JWT Authentication & RBAC Middleware

Implement the full authentication and authorization layer for Crudier CRM supporting seven roles: Founder, Admin, Team Lead, Developer, Designer, Sales, and Intern. Use a dual-token strategy: a 15-minute access token and a 7-day refresh token, both signed with environment-stored secrets.

Expose these endpoints under /api/v1/auth: POST /register, POST /login, POST /logout, GET /me, and POST /refresh-token. Hash passwords with bcryptjs at 12 salt rounds. On login, issue both tokens — deliver the refresh token as an HttpOnly Secure cookie and the access token in the JSON body.

The user schema must include: name, email, password, role, workspaceId, avatar, isActive, createdAt, and updatedAt. Use MongoDB native driver only — no Mongoose. Build a protectRoute middleware that verifies the Bearer token and attaches req.user. Build an authorizeRoles(...roles) factory middleware returning 403 for unauthorized roles. Handle edge cases: duplicate email, wrong password, expired token, and inactive account. Write all queries in userRepository.js.

Key Deliverables

✓ Register, Login, Logout, /me, Refresh Token endpoints
✓ bcryptjs password hashing (12 rounds)
✓ Dual JWT strategy (access + HttpOnly refresh cookie)
✓ protectRoute and authorizeRoles middleware
✓ userRepository.js with MongoDB queries
✓ Input validation and structured 4xx error responses

Develop Multi-Workspace Management Module

Build the Workspace Management module — Crudier’s multi-tenancy backbone. Each workspace represents a startup or team, and users can belong to multiple workspaces with different roles in each.

The workspace schema includes: name, description, logo, ownerId, members [{userId, role, joinedAt}], settings, and createdAt. Expose endpoints under /api/v1/workspaces: POST / (create), GET /:id, PATCH /:id, DELETE /:id (Founder only), POST /:id/invite, PATCH /:id/members/:userId/role, DELETE /:id/members/:userId, and GET /:id/activity.

Enforce role-based access at the route level: only Founders can delete workspaces or reassign roles; Admins can invite and remove members. Implement a PATCH /switch endpoint that updates the user’s active workspaceId. Log every significant action (invite, role change, settings update) to a workspaceActivity collection with: workspaceId, actorId, action, targetId, timestamp. Expose activity via GET /:id/activity with pagination. Write all queries in workspaceRepository.js. Protect all endpoints with protectRoute from CRUD-002.

Key Deliverables

✓ Full workspace CRUD endpoints with role guards
✓ Member invite, role update, and removal endpoints
✓ workspaceMiddleware for membership validation
✓ Activity log collection with paginated GET endpoint
✓ switchWorkspace endpoint
✓ workspaceRepository.js

Build Team Member Management & Profile System

Build the Team Management module for managing all workspace members. Expose endpoints under `/api/v1/team`: GET `/` (list with pagination & search), GET `/:userId` (profile), PATCH `/:userId/profile`, PATCH `/:userId/role` (Admin/Founder only), PATCH `/:userId/department`, PATCH `/:userId/deactivate` (Admin/Founder only), GET `/:userId/activity`, and POST `/:userId/avatar`.

The list endpoint must support: case-insensitive name/email search, filtering by role, department, and `isActive`, pagination via page and limit, and sorting by name or `createdAt`. Return total count alongside results.

Avatar uploads use `multer` with `diskStorage` under `/uploads/avatars/`. Validate file type (jpeg/png/webp) and cap size at 2 MB. Store the relative URL in the user's avatar field. The skills field (string array) is editable via PATCH `/profile`. Deactivation sets `isActive` to false and records `deactivatedAt`; deactivated users cannot log in. All queries must be workspace-scoped. Write queries in `teamRepository.js`.

Key Deliverables

- | |
|--|
| ✓ Paginated member list with search and filter |
| ✓ Profile update, role, and department endpoints |
| ✓ Multer avatar upload (type + size validation) |
| ✓ Soft deactivation with <code>deactivatedAt</code> timestamp |
| ✓ Skills array management via profile update |
| ✓ <code>teamRepository.js</code> with workspace-scoped queries |

Engineer Enterprise-Grade Task Management System

Build the core task lifecycle system for Crudier CRM. The task schema: title, description, assignedTo, assignedBy, workspaceId, priority (Low/Medium/High/Critical), status (Pending/In Progress/Submitted/Review/Approved/Rejected/Blocked), deadline, attachments, comments [{userId, text, createdAt}], and activityLogs [{action, actorId, timestamp}].

Endpoints under /api/v1/tasks: POST / (create), GET / (list), GET /:id, PATCH /:id, DELETE /:id (Admin/Founder), POST /:id/comments, DELETE /:id/comments/:cid, POST /:id/attachments (multer), PATCH /:id/status, and GET /:id/history.

Status transitions are role-gated: Developers/Designers → In Progress, Submitted; Team Leads → Review, Approved, Rejected; Founders/Admins → Blocked. Every status change auto-writes to activityLogs. The list endpoint supports filters: assignedTo, priority, status, deadline range, and title text search; plus sorting and offset pagination. Separate business logic (status guards, log writing) in taskService.js from queries in taskRepository.js.

Key Deliverables

- | |
|---|
| ✓ Full task CRUD with role-gated status transitions |
| ✓ Comment add/delete endpoints |
| ✓ Multer attachment upload under /uploads/tasks/ |
| ✓ Auto activityLogs on every status change |
| ✓ Advanced list filtering, sorting, and pagination |
| ✓ taskRepository.js and taskService.js separation |

Build Attendance Tracking & Analytics System

Implement the Attendance Management System for daily check-ins, check-outs, hour calculations, and productivity analytics.

Schema: `userId`, `workspaceId`, `checkIn`, `checkOut`, `totalHours` (float), `status` (Present/Late/Absent/Half Day), `date` (YYYY-MM-DD). Endpoints under `/api/v1/attendance`: `POST /checkin`, `POST /checkout`, `GET /today`, `GET /history` (paginated, date-range filter), `GET /report` (Admin, workspace-wide), `GET /analytics`.

Business logic: prevent duplicate check-ins per day. Auto-assign status on check-in — before 9:30 AM → Present; 9:30–10:30 AM → Late. On checkout, compute `totalHours`; if under 4 hours and status was Present, downgrade to Half Day. The `/report` endpoint returns per-user breakdowns (present, absent, late, half day, avg hours) using MongoDB `$group` aggregation. The `/analytics` endpoint returns workspace attendance rate, punctuality rate, and a 30-day trend array. Protect Admin endpoints with `authorizeRoles`. Write all queries in `attendanceRepository.js` with documented pipeline comments.

Key Deliverables

✓ Check-in / check-out with duplicate prevention
✓ Auto-status logic (Present / Late / Half Day)
✓ <code>totalHours</code> calculation on checkout
✓ Paginated history with date-range filter
✓ Admin <code>/report</code> using MongoDB aggregation
✓ Analytics endpoint with 30-day trend

Develop Notes, Documentation & Full-Text Search

Build the Notes and Documentation module — Crudier's internal knowledge base for meeting notes, startup docs, and tagged references.

Schema: title, content (Markdown string), createdBy, workspaceId, tags (string array), visibility (private/workspace/team), archived (bool), createdAt, updatedAt. Endpoints under /api/v1/notes: POST /, GET /, GET /:id, PATCH /:id, DELETE /:id (creator or Admin), POST /:id/archive, POST /:id/restore, POST /:id/comments, DELETE /:id/comments/:cid, and GET /search.

The list endpoint filters by tags, visibility, archived status, and createdBy, with pagination and newest-first sorting. For GET /search?q=, use a MongoDB text index on title and content, created on app startup. Return results sorted by \$meta textScore. Enforce visibility rules in the list query: private → creator only; workspace → all members; team → same department. Archived notes are excluded by default but accessible with ?archived=true. Write all queries in notesRepository.js.

Key Deliverables

✓ Full notes CRUD endpoints
✓ Text index on title + content; /search with relevance sort
✓ Archive / restore endpoints
✓ Visibility rules enforced in list query
✓ Tag filtering and pagination
✓ Comment add/delete on notes

Build Meeting Scheduling & Calendar API

Implement the Meeting and Scheduling module for Crudier CRM.

Schema: title, description, organizerId, participants [{userId, rsvp}], startTime, endTime, meetingLink (URL), workspaceId, notes, status (Scheduled/In Progress/Completed/Cancelled), createdAt. Endpoints under /api/v1/meetings: POST /, GET /, GET /:id, PATCH /:id (organizer/Admin), DELETE /:id (cancel — organizer/Admin), POST /:id/rsvp, POST /:id/notes (organizer only), GET /upcoming, and GET /:id/attendance.

Validation: startTime must be future on creation; endTime must be after startTime; meetingLink must pass URL regex; all participants must be workspace members. The /upcoming endpoint returns the next 7 days of meetings for the authenticated user (as organizer or participant), sorted chronologically. On cancellation, call createNotification() from CRUD-009 for all participants — coordinate on the calling interface early. Protect organizer-only endpoints via role-check middleware. Write all queries in meetingRepository.js.

Key Deliverables

✓ Full meeting CRUD with validation
✓ RSVP endpoint for participants
✓ Post-meeting notes endpoint (organizer only)
✓ GET /upcoming (next 7 days)
✓ Cancellation flow with notification hook
✓ meetingRepository.js with all queries

Build Scalable Notification & Alert System

Build the Notification System that powers alerts across all Crudier modules and is importable by other services.

Schema: `userId`, `title`, `message`, `type` (`task_assigned` / `meeting_reminder` / `attendance_alert` / `announcement` / `submission_approved`), `isRead` (bool, default `false`), `metadata` (object), `createdAt`. Endpoints under `/api/v1/notifications`: `GET /`, `GET /unread-count`, `PATCH /:id/read`, `PATCH /read-all`, `DELETE /:id`, `DELETE /clear-all`.

Export an async `createNotification(userId, {title, message, type, metadata})` utility from `notificationService.js`. This is called by `taskService` (task assigned/approved), `meetingService` (cancellation), and `attendanceService` (late/absent). Coordinate with engineers CRUD-005, CRUD-006, and CRUD-008 on the calling interface. The list endpoint filters by `type` and `isRead`, with pagination. The `/unread-count` endpoint uses `countDocuments` with a compound `{userId, isRead}` index for speed. Group consecutive same-type notifications within 1 hour at the service layer (not in MongoDB). Create `userId` and `createdAt` indexes on startup.

Key Deliverables

- | |
|---|
| ✓ Full notification REST endpoints |
| ✓ Exported <code>createNotification()</code> utility in <code>notificationService.js</code> |
| ✓ Unread count endpoint with compound index |
| ✓ Grouping logic at service layer |
| ✓ Type + <code>isRead</code> filtering with pagination |
| ✓ Collection indexes created on app startup |

Build Sales CRM Lead & Pipeline Management

Build the Sales CRM module managing leads from first contact to closed deal.

Lead schema: companyName, contactPerson, email, phone, assignedTo, workspaceId, status (New Lead / Contacted / Meeting Scheduled / Negotiation / Closed Won / Closed Lost), estimatedValue, notes [{text, createdBy, createdAt}], followUpDate, proposalUrl, createdAt. Endpoints under /api/v1/sales: POST /leads, GET /leads (filtered list), GET /leads/:id, PATCH /leads/:id, DELETE /leads/:id (Admin/Founder), POST /leads/:id/notes, PATCH /leads/:id/status, GET /pipeline, GET /analytics, and GET /follow-ups.

The /pipeline endpoint groups leads by status using MongoDB \$group, returning count and total estimatedValue per stage. The /analytics endpoint returns: total leads, conversion rate (Closed Won ÷ total), total estimated revenue, average deal value, win/loss ratio, and a 6-month new-leads trend — all via aggregation pipelines. The /follow-ups endpoint returns leads with followUpDate within the next 7 days. Create a text index on companyName and contactPerson for search. Restrict module access to Sales role and above via authorizeRoles. Write queries in salesRepository.js and logic in salesService.js.

Key Deliverables

- | |
|--|
| ✓ Full lead CRUD with role guard |
| ✓ /pipeline endpoint with \$group aggregation |
| ✓ /analytics with conversion and revenue metrics |
| ✓ /follow-ups for upcoming 7 days |
| ✓ Text search on companyName and contactPerson |
| ✓ salesRepository.js and salesService.js |